

category: avalonia

tags: [avalonia, shutdown, ShutdownRequested, IHost, StopAsync, deadlock, sync-over-async, generic-host, kestrel, stale-binary]

severity: high

created: 2026-07-26

updated: 2026-07-26

project-origin: App-Access-V2 (migrate-avalonia STEP-2.3)

Sync-block `IHost.StopAsync()` trong `ShutdownRequested` của Avalonia → deadlock, app không bao giờ thoát

Tình huống gặp phải

App Avalonia self-host ASP.NET Core (Kestrel) bên trong (host model thay cho bản WinForms lai `Microsoft.NET.Sdk.Web`). Cần dừng `IHost` + RabbitMQ trước khi app thoát (nếu không tiến trình không thoát — ADR C1). Cách "hiển nhiên":

```
desktop.ShutdownRequested += (_, _) =>
{
    apiHost.StopAsync().ConfigureAwait(false).GetAwaiter().GetResult(); // ← DEADLOCK
};
```

Triệu chứng / Lỗi

- Đóng window: Kestrel unbind xong (port đóng, netstat sạch) nhưng **process sống mãi** (>60s), `Responding=True`, không window.
- `dotnet-dump` main thread cho thấy block vĩnh viễn:

```
System.Threading.SynchronizationContext.WaitHelper(...)
Avalonia.Threading.AvaloniaSynchronizationContext.Wait(...)
System.Threading.Tasks.Task.SpinnableWait(...)
App.ShutdownServices() ← GetAwaiter().GetResult()
ClassicDesktopStyleApplicationLifetime.DoShutdown(...) → HandleWindowClosed → WndProc
```

Nguyên nhân gốc rễ (Root Cause)

`ShutdownRequested` chạy trên UI thread (bên trong `DoShutdown`, giữa chuỗi `WndProc` `HandleClosed`).

Sync-block UI thread bằng `GetResult()` trong khi Task `IHost.StopAsync()` có continuation cần được hoàn tất qua ngữ cảnh đã bị block (`AvaloniaSynchronizationContext`) → sync-over-async deadlock kinh điển. Kestrel dừng được một phần (port đóng) nhưng Task tổng không bao giờ complete → UI thread chờ mãi → `DoShutdown` không kết thúc → `Main` không return.

Giải pháp

Pattern **cancel-first async shutdown** (verify chạy thật OK):

```
private static bool shutdownHandled;
desktop.ShutdownRequested += async (_, e) =>
{
    if (shutdownHandled) return; // lần 2 do Shutdown() bên dưới → cho thoát thật
    shutdownHandled = true;
    e.Cancel = true; // PHẢI set TRƯỚC await đầu tiên
    try { eventBus?.Dispose(); } catch { }
    var stopTask = Task.Run(() => apiHost.StopAsync()); // stop hoàn toàn trên
    threadpool
    await Task.WhenAny(stopTask, Task.Delay(TimeSpan.FromSeconds(10))); // timeout an
    toàn
    desktop.Shutdown(); // trigger ShutdownRequested lần 2 → flag cho qua
};
```

Kết quả đo thực tế: `/health 200` lúc chạy; `CloseMainWindow()` → process thoát sạch < 8s, port nhả.

Áp dụng lại (How to reuse)

- Mọi app Avalonia có host/service nền (IHost, Kestrel, RabbitMQ, gRPC...) cần dọn khi thoát → dùng pattern cancel-first ở trên, KHÔNG bao giờ `GetResult()` / `.Wait()` trong `ShutdownRequested`.
- `e.Cancel = true` phải đặt trước `await` đầu tiên (sau `await`, event args đã hết tác dụng).
- Luôn kèm timeout (`Task.WhenAny + Delay`) — nếu host kẹt vẫn phải cho app thoát (parity `Environment.Exit(0)` của bản WinForms nguồn).

Chú ý / Cạm bẫy (Gotchas)

- ⚠ Bẫy stale-binary khi debug vụ này: process treo giữ lock file exe → lần test sau `Start-Process` chạy **binary CŨ** (fix chưa vào) → tưởng fix không ăn. PHẢI `Stop-Process -Name <app> -Force` TRƯỚC mỗi vòng rebuild+retest, và kiểm tra build log có `0 Error` + không có lỗi copy apphost.
- ⚠ Port đóng ≠ host stop xong ≠ process thoát — 3 mức khác nhau, phải verify đủ 3 (netstat + Task complete + Get-Process rỗng).
- ⚠ `async void`-style handler trên `ShutdownRequested` hợp lệ ở đây vì có flag chống re-entry; không có flag sẽ loop vô hạn Cancel/Shutdown.

Tham chiếu

- File: `iAccessDesktopv2.Avalonia/App.axaml.cs` (pattern đã fix), `iAccess.Core/Hosting/ApiHost.cs`
- Dump bằng chứng: `temp/migrate-avalonia-step23/hang.dmp`, `hang2.dmp` (git-ignored)
- Project: App-Access-V2, plan `PLAN-migrate-avalonia-2026-07-26 STEP-2.3`